

Project 6: File System Manager

Nolan Tuttle & Mathew Hobson

College of Engineering and Technology, Grand Canyon University

CST-315: Operating Systems Lecture and Lab

Professor Citro

April 16, 2025

Updated Description

CitroShell is a Linux-based command line interpreter with functioning support for most Linux shell commands. In this iteration of our interpreter, we focused on creating support for file management, including an `rm`, `mkdir`, `touch`, and `rmdir` command.

Methodology/Approach

1. Refactored Code Structure: Slightly improved initial user prompts and readability.
2. New Features:
 - Remove Directory (`rmdir`): Implemented the ability to remove directories.
 - Directory Creation (`mkdir`): Implemented to allow for folder creation using `execv()` so that they can be navigated using `cd`.
 - Remove Files (`rm`): Implemented the ability to remove files.
 - File Creation (`touch`): Implemented the ability to create files.

Algorithm for Parsing and Processing Shell Commands

1. Read user input, removing trailing newline characters
2. Check for special characters (`cd` for directory changes, `|` for pipes).
3. Tokenize the input based on semicolon (`;`).
4. Handle background commands by using `fork()`.
5. Execute commands using `execv()` instead of `system()` for better security and efficiency.
6. Repeat until user exits the shell.

Processing Input

```
int isRunning = 1; // Running condition for command line

printf("<CTRL+X to end execution>\n"); // Initial prompt
printf("<CTRL+B to exit the shell>\n\n"); // Initial prompt
char str[100]; // Allocate space for input string
char currDirectory[100];
while (isRunning)
{
    printf("citroShell$ "); // Command line prompt
    fgets(str, sizeof(str), stdin); // Read input, including spaces
    str[strcspn(str, "\n")] = 0; // Remove newline character

    // Condition for user inputting "CTRL+X" or the "quit" command
    if (str[0] == 24 || (!strcmp(str, "quit")))
    {
        exit(1); // Ends execution
    }
    // Condition for user inputting "CTRL+B" or the "exit" command
    else if (str[0] == 2 || (!strcmp(str, "exit")))
    {
        isRunning = 0;
        break;
    }

    // splits up the input into semicolon-separated tokens
    char *command = strtok(str, ";");
```

Gets user's string input, removing the newline character and keeping all spaces, then tokenizing it using semicolons for later processing.

Command Execution

```
char *command = strtok(str, ";");

while (command != NULL)
{
    if (command[0] == 'c' && command[1] == 'd')
    {
        char *temp = strtok(command, " ");
        temp = strtok(NULL, " ");
        chdir(temp);
        break;
    }

    char *args[] = {"/bin/bash", "-c", command, NULL};

    pid_t pid = fork(); // create a child process

    if (pid == 0)
    {
        if (execv("/bin/bash", args) == -1)
        {
            perror("exec failed");
            exit(1);
        }
    }
    else if (pid > 0)
    {
        int status;
        pid_t childPID = waitpid(pid, &status, 0);
    }
    command = strtok(NULL, ";");
}
```

Creating child processes using fork() and executing bash commands using execv() in a loop that runs until no more commands are given.

References

Stallings, W. (2019). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.

Tuttle, N., Hobson, M., (2025) *CST315* GitHub. <https://github.com/nolantuttle/CST315.git>